

CONTENTS

Módulo 5 — Ejercicios Adicionales de Práctica	
Von Neumann y Organización de la CPU — Arquitectura de Computadoras, FING UdelaR	
Índice de dificultad	
0. Especificación de ISA-P (necesaria para todos los ejercicios)	
Ejercicio 1 (Fácil) — Traza fetch-decode-execute sin memoria	
Resolución	
Ejercicio 2 (Medio) — Traza fetch-decode-execute con memoria y dos modos de direccionamiento	
Resolución	
Ejercicio 3 (Fácil-Medio) — Codificación de instrucciones (aritmética + memoria directa)	
Resolución	
Ejercicio 4 (Medio) — Codificación de instrucciones (lógicas + salto condicional)	
Resolución	
Ejercicio 5 (Medio) — Traducción de un if / else	
Resolución	
Ejercicio 6 (Difícil) — Traducción de un while con recorrido de arreglo	
Resolución	
Ejercicio 7 (Medio-Difícil) — Decodificación e interpretación de código máquina dado	
Resolución	
Preguntas teóricas rápidas tipo parcial	

Resumen de técnicas usadas en este documento

Para el examen

Módulo 5 — Ejercicios Adicionales de Práctica

VON NEUMANN Y ORGANIZACIÓN DE LA CPU — ARQUITECTURA DE COMPUTADORAS, FING UDELAR

⚠ Nota de método (léase antes de resolver): estos ejercicios son **nuevos**, no una repetición de la Guía Temática ni del Práctico 6 ya resueltos. Para poder plantear ejercicios de codificación y traducción autocontenidos se definió una ISA de práctica propia para este documento, **ISA-P** (distinta del formato usado en el Práctico 6, tanto en el opcode como en la disposición de campos), con modos de direccionamiento explícitos en el propio formato — algo que el Práctico 6 no ejercitaba directamente. La especificación completa de ISA-P está en la Sección 0.

Verificación: cada traza de registros, cada codificación binaria/hex y cada traducción de código de este documento se verificó ejecutando un ensamblador + simulador de ISA-P escrito en Python para esta resolución (codificador `enc_*`, decodificador `decode()` y una CPU con ciclo fetch-decode-execute completo, incluyendo PC, IR, MAR, MBR y flags Z/N). Ningún valor de este documento fue calculado “a mano y de memoria” sin ese chequeo automatizado.

Índice de dificultad

#	Ejercicio	Tema	Dificultad
1	Traza fetch-decode-execute (sin memoria)	Ciclo de instrucción, registros parcial/totalmente visibles	Fácil
2	Traza fetch-decode-execute (con memoria, 2 modos de direccionamiento)	Ciclo de instrucción, MAR/MBR, direccionamiento	Medio
3	Codificación de instrucciones — aritmética + memoria directa	Formato de instrucción, cálculo de bits	Fácil-Medio
4	Codificación de instrucciones — lógicas + salto condicional	Formato de instrucción, flags	Medio
5	Traducción de un <code>if / else</code> a ISA-P	Traducción de alto nivel a ensamblador	Medio
6	Traducción de un <code>while</code> con recorrido de arreglo	Traducción de alto nivel, direccionamiento indirecto	Difícil
7	Decodificación e interpretación de código máquina dado	Análisis inverso (ingeniería inversa de binario a semántica)	Medio-Difícil

0. Especificación de ISA-P (necesaria para todos los ejercicios)

Arquitectura: 16 bits, registros de uso general R0-R7 (8 registros), memoria unificada de datos e instrucciones (Von Neumann puro — no Harvard), banco de registros parcialmente visible: **PC** (Program Counter) y flags **Z** (zero) y **N** (negative, bit de signo del resultado). Registros internos (no visibles al programador): **IR** (Instruction Register), **MAR** (Memory Address Register), **MBR** (Memory Buffer Register, también llamado MDR).

Set de instrucciones: 12 instrucciones. Bits de opcode: $\lceil \log_2(12) \rceil = 4$ bits (verificación: $2^4=16 \geq 12$; sobran 4 códigos sin asignar, lo cual es normal). Bits de registro: $\lceil \log_2(8) \rceil = 3$ bits (verificación: $2^3=8$, exacto).

Opcode (bin)	Instrucción	Opcode (bin)	Instrucción
0000	NOP	0110	CMP
0001	MOVI	0111	LOAD
0010	ADD	1000	STORE
0011	SUB	1001	JMP
0100	AND	1010	JZ
0101	OR	1011	JN

Registros: R0=000, R1=001, ..., R7=111 (binario de 3 bits del número).

Todas las instrucciones ocupan **16 bits fijos** (ISA-P es RISC: formato fijo, load-store). Cinco sub-formatos, cada uno verificado por suma de anchos = 16:

Formato	Instrucciones	Bits 15-12	Bits 11-9	Bits 8	Bits 7-0 / 6-0
Sin operandos	NOP	opcode(4)	—	—	sin usar (12 bits)
Inmediato	MOVI Rd,Imm	opcode(4)	Rd(3)	Imm (9 bits, bits 8-0)	
3 registros	ADD/SUB/AND/OR Rd,Rs1,Rs2	opcode(4)	Rd(3)	Rs1(3)	Rs2(3) + sin usar(3)
Comparación	CMP Rs1,Rs2	opcode(4)	sin usar(3)	Rs1(3)	Rs2(3) + sin usar(3)
Memoria	LOAD/STORE Rx, [Dir] o Rx,(Rb)	opcode(4)	Rx(3)	Modo(1)	Dir(8) o [sin usar(5)+Reg(3)]
Salto	JMP/JZ/JN Dir	opcode(4)	sin usar(4)		Dir(8)

Verificación de anchos: Inmediato $4+3+9=16 \checkmark$ · 3 registros $4+3+3+3+3=16 \checkmark$ · Comparación $4+3+3+3+3=16 \checkmark$ (el primer campo de 3 bits no se usa: CMP no escribe resultado, solo actualiza Z/N) · Memoria $4+3+1+8=16 \checkmark$ · Salto $4+4+8=16 \checkmark$.

Modos de direccionamiento de LOAD/STORE (campo Modo, 1 bit — $\lceil \log_2(2) \rceil = 1$, exacto):

- **Modo 0 — directo a memoria:** el campo Dir (8 bits) es la dirección de memoria en sí (rango 0-255). `LOAD R1, [0x21]` $\Rightarrow R1 = \text{mem}[0x21]$.
- **Modo 1 — indirecto a registro:** los 3 bits menos significativos del campo de 8 bits indican un registro que contiene la dirección real (los 5 bits restantes del campo no se usan). `LOAD R1, (R0)` $\Rightarrow R1 = \text{mem}[R0]$.

Semántica de las instrucciones:

- `MOVI Rd, Imm`: $Rd = \text{Imm}$ (Imm de 9 bits sin signo, 0-511).
- `ADD/SUB/AND/OR Rd, Rs1, Rs2`: $Rd = Rs1 \text{ op } Rs2$. Actualiza Z (resultado==0) y N (bit de signo del resultado, complemento a 2 de 16 bits).
- `CMP Rs1, Rs2`: calcula $Rs1 - Rs2$ internamente (no se guarda en ningún registro visible), solo actualiza Z y N. Es la única forma de comparar en ISA-P (no hay instrucción de comparación con inmediato).
- `LOAD Rx, ...`: $Rx = \text{mem}[\text{dirección resuelta según el modo}]$.
- `STORE Rx, ...`: $\text{mem}[\text{dirección resuelta según el modo}] = Rx$.
- `JMP Dir`: $PC = \text{Dir}$ (salto incondicional).
- `JZ Dir`: si $Z == 1$, $PC = \text{Dir}$ (salta si el último ADD/SUB/AND/OR/CMP dio 0, o sea, si los dos operandos eran iguales en el caso de CMP).
- `JN Dir`: si $N == 1$, $PC = \text{Dir}$ (salta si el último resultado fue negativo, o sea, en un CMP $Rs1, Rs2$, si $Rs1 < Rs2$).

⚠ No hay instrucción de copia registro-a-registro dedicada. Cuando un ejercicio necesita copiar un valor sin pasar por memoria, se usa el modismo estándar de ISAs RISC sin instrucción MOV: `ADD Rdst, Rsrc, Rcero` con un registro previamente puesto a 0 (o, si el valor es una constante conocida, directamente `MOVI`).

Ejercicio 1 (Fácil) — Traza fetch-decode-execute sin memoria

Enunciado: dado el siguiente programa en ISA-P, ubicado en memoria a partir de la dirección 0, completar la traza de PC, IR, MAR, MBR, flags Z/N y banco de registros **después de cada instrucción completa** (Fetch+Decode+Read+Execute+Write).

```

0: MOVI R0, 10
1: MOVI R1, 4
2: SUB  R2, R0, R1
3: CMP  R2, R1
4: JZ   6
5: MOVI R3, 99
6: NOP

```

RESOLUCIÓN

Codificación de cada instrucción (verificada con el codificador de ISA-P):

Dir	Instrucción	Binario (16 bits)	Hex
0	MOVI R0,10	0001 000 000001010	0x100A
1	MOVI R1,4	0001 001 000000100	0x1204
2	SUB R2,R0,R1	0011 010 000 001 000	0x3408
3	CMP R2,R1	0110 000 010 001 000	0x6088
4	JZ 6	1010 0000 00000110	0xA006
5	MOVI R3,99	0001 011 001100011	0x1663
6	NOP	0000 0000 00000000	0x0000

Traza paso a paso (estado **al terminar** cada instrucción; PC ya apunta a la siguiente por el incremento del Fetch):

Paso	Instr. ejecutada	PC (nuevo)	MAR	MBR (=IR leído)	Z	N	R0	R1	R2	R3
1	MOVI R0,10	1	0	0x100A	0	0	10	0	0	0
2	MOVI R1,4	2	1	0x1204	0	0	10	4	0	0
3	SUB R2,R0,R1	3	2	0x3408	0	0	10	4	6	0
4	CMP R2,R1	4	3	0x6088	0	0	10	4	6	0
5	JZ 6	5 (no salta)	4	0xA006	0	0	10	4	6	0
6	MOVI R3,99	6	5	0x1663	0	0	10	4	6	99
7	NOP	7	6	0x0000	0	0	10	4	6	99

Razonamiento del salto (paso 5): `CMP R2,R1` calculó internamente $R2-R1=6-4=2 \neq 0$, así que Z quedó en 0. Por lo tanto `JZ 6` **no** se toma, y el PC simplemente sigue en secuencia hacia la instrucción 5 (`MOVI R3,99`), que sí se ejecuta.

Estado final: R0=10, R1=4, R2=6, R3=99, Z=0, N=0, PC=7.

✅ **Verificación:** se simuló este programa con la CPU de ISA-P (fetch real usando MAR=PC, IR=mem[MAR], PC++, decode, execute). El estado final de registros coincidió exactamente con la tabla: `R = [10, 4, 6, 99, 0, 0, 0, 0]`, `Z=0`, `N=0`. Se verificó además que, si `CMP` se hubiera hecho entre R2 y un registro igual a 6 (en vez de R1=4), Z habría dado 1 y el `JZ 6` sí se habría tomado, saltando directo a NOP sin ejecutar `MOVI R3,99` — confirmando que el mecanismo de salto condicional depende exclusivamente del flag Z fijado por la instrucción *inmediatamente anterior*.

Ejercicio 2 (Medio) — Traza fetch-decode-execute con memoria y dos modos de direccionamiento

Enunciado: estado inicial de memoria: `mem[0x20] = 7` (el resto de la memoria en 0, todos los registros en 0). Programa:


```

0: MOVI R0, 0x20
1: LOAD R1, (R0)      ; indirecto a registro
2: MOVI R2, 3
3: ADD  R3, R1, R2
4: STORE R3, [0x21]    ; directo a memoria
5: LOAD R4, [0x21]    ; directo a memoria

```

Completar la traza de **MAR y MBR** en cada instrucción que toca memoria, y el banco de registros al final de cada instrucción.

RESOLUCIÓN

Codificación (verificada):

Dir	Instrucción	Hex
0	MOVI R0,0x20	0x1020
1	LOAD R1,(R0)	0x7300
2	MOVI R2,3	0x1403
3	ADD R3,R1,R2	0x2650
4	STORE R3,[0x21]	0x8621
5	LOAD R4,[0x21]	0x7821

Traza:

Paso	Instr.	¿Toca memoria?	MAR	MBR	R0	R1	R2	R3	R4
1	MOVI R0,0x20	No (solo fetch de la instrucción, MAR=0)	0	0x1020	0x20	0	0	0	0
2	LOAD R1, (R0)	Sí: modo indirecto a registro → dirección = R0 = 0x20	0x20	7	0x20	7	0	0	0
3	MOVI R2,3	No	2	0x1403	0x20	7	3	0	0
4	ADD R3,R1,R2	No (operandos ya en registros)	3	0x2650	0x20	7	3	10	0
5	STORE R3,[0x21]	Sí: modo directo a memoria → dirección = 0x21 (campo Dir de la propia instrucción)	0x21	10	0x20	7	3	10	0
6	LOAD R4, [0x21]	Sí: modo directo a memoria → dirección = 0x21	0x21	10	0x20	7	3	10	10

Notar la diferencia clave entre el paso 2 y los pasos 5-6: en el paso 2 (**indirecto a registro**), la dirección efectiva (0x20) sale de leer el registro R0 — si R0 cambiara antes de re-ejecutar esta instrucción, apuntaría a otra celda. En los pasos 5-6 (**directo a memoria**), la dirección 0x21 está fija dentro del propio código de la instrucción y nunca cambia sin reensamblar el programa.

Estado final de memoria: mem[0x20]=7 (sin cambios), mem[0x21]=10 (escrito en el paso 5).

✅ **Verificación:** se simuló el programa completo con la CPU de ISA-P, inicializando

mem[0x20]=7. El estado final de registros coincidió exactamente: R = [0x20, 7, 3, 10, 10, 0, 0, 0], y mem[0x21]=10. Se confirmó además, ejecutando una segunda pasada con mem[0x20]=50 en lugar de 7, que el resultado final cambia proporcionalmente (R4 termina en 53 = 50+3), lo que confirma que el mecanismo de lectura por MAR/MBR está correctamente modelado y no hardcodeado.

Ejercicio 3 (Fácil-Medio) — Codificación de instrucciones (aritmética + memoria directa)

Enunciado: usando el formato de ISA-P (Sección 0), codificar en binario de 16 bits y en hexadecimal las siguientes instrucciones:

```

MOVI  R2, 0x15
MOVI  R5, 0x03
ADD   R1, R2, R5
LOAD  R6, [0x40]
STORE R1, (R6)

```

RESOLUCIÓN

Instrucción	Opcode	Campos	Binario (16 bits)	Hex
MOVI R2, 0x15	0001	Rd=010, Imm=0 0001 0101	0001 010 000010101	0x1415
MOVI R5, 0x03	0001	Rd=101, Imm=0 0000 0011	0001 101 000000011	0x1A03
ADD R1, R2, R5	0010	Rd=001, Rs1=010, Rs2=101, sin usar=000	0010 001 010 101 000	0x22A8
LOAD R6, [0x40]	0111	Rx=110, Modo=0, Dir=0100 0000	0111 110 0 01000000	0x7C40
STORE R1, (R6)	1000	Rx=001 (registro con el <i>valor</i> a guardar), Modo=1, campo=00000 110 (110=R6)	1000 001 1 00000110	0x8306

Detalle del cálculo de `STORE R1, (R6)`, por ser el más propenso a error: la instrucción `STORE` siempre coloca en el campo `Rx` (bits 11-9) el registro **cuyo valor se va a guardar**, nunca el registro base de la dirección. Como el modo es 1 (indirecto a registro), el campo de 8 bits no es una dirección — sus 3 bits menos significativos codifican **qué registro contiene** la dirección de destino (aquí, R6 = binario 110); los 5 bits restantes de ese campo se dejan en 0 por convención.

✅ **Verificación:** las cinco codificaciones se generaron con el codificador de ISA-P (`enc_movi`, `enc_reg3`, `enc_mem`) y coincidieron con la tabla. Se verificó además, decodificando cada hex de vuelta con el decodificador, que se recupera exactamente la instrucción original en los cinco casos (round-trip encode→decode sin pérdida).

Ejercicio 4 (Medio) — Codificación de instrucciones (lógicas + salto condicional)

Enunciado: codificar la siguiente secuencia (nótese que incluye una instrucción `CMP`, que no escribe resultado, y un salto condicional cuyo destino es una dirección absoluta):

```
MOVI R0, 0x0A
MOVI R7, 0x01
SUB R3, R0, R7
AND R4, R0, R3
STORE R4, [0x50]
CMP R3, R7
JZ 0x0A
```

RESOLUCIÓN

Instrucción	Opcode	Campos	Binario (16 bits)	Hex
MOVI R0,0x0A	0001	Rd=000, Imm=0 0000 1010	0001 000 000001010	0x100A
MOVI R7,0x01	0001	Rd=111, Imm=0 0000 0001	0001 111 000000001	0x1E01
SUB R3,R0,R7	0011	Rd=011, Rs1=000, Rs2=111, sin usar=000	0011 011 000 111 000	0x3638
AND R4,R0,R3	0100	Rd=100, Rs1=000, Rs2=011, sin usar=000	0100 100 000 011 000	0x4818
STORE R4, [0x50]	1000	Rx=100, Modo=0, Dir=0101 0000	1000 100 0 01010000	0x8850
CMP R3,R7	0110	sin usar=000, Rs1=011, Rs2=111, sin usar=000	0110 000 011 111 000	0x60F8
JZ 0x0A	1010	sin usar=0000, Dir=0000 1010	1010 0000 00001010	0xA00A

Verificación semántica adicional (para que el ejercicio no sea solo mecánico): con R0=10 y R7=1, `SUB R3,R0,R7` da R3=9; `AND R4,R0,R3` = 10 AND 9 = $1010 \text{ AND } 1001 = 1000$ = 8, así que R4 termina en 8. Luego `CMP R3,R7` calcula $9-1=8 \neq 0$ → Z=0, por lo tanto el `JZ 0x0A`

final **no** se tomaría si se ejecutara este fragmento — es un buen ejemplo de que codificar bien una instrucción y que esa instrucción efectivamente altere el flujo del programa son dos preguntas distintas.

✅ **Verificación:** las siete codificaciones se generaron y confirmaron con el codificador de ISA-P. Adicionalmente se ejecutó el fragmento completo en la CPU simulada (partiendo de R0=10 y R7=1 puestos por `MOVI`): dio R3=9, R4=8, Z=0 tras el `CMP`, exactamente como el cálculo manual de arriba.

Ejercicio 5 (Medio) — Traducción de un `if / else`

Enunciado: traducir a ISA-P el siguiente fragmento, usando R0 para `a`, R1 para `b` y R2 para `resultado`:

```
if (a > b) {
    resultado = a - b;
} else {
    resultado = 0;
}
```

RESOLUCIÓN

ISA-P no tiene una instrucción “salta si mayor”, así que hay que construir la condición con `CMP` + `JN` (salta si el resultado de la resta fue negativo). La idea: `CMP a,b` calcula $a-b$; si el resultado es negativo, entonces $a < b$ (o sea, **no** se cumple $a > b$) y hay que ir directo a la rama `else`.

```
0: MOVI R0, <a>          ; carga de prueba, ver casos abajo
1: MOVI R1, <b>
2: CMP  R0, R1           ; calcula a-b, fija Z/N
3: JN   6                ; si a-b < 0 (a<b) -> saltar a "else" (dir. 6)
4: SUB  R2, R0, R1       ; rama "then": resultado = a-b
5: JMP  7                ; saltar al final, evitando caer en el "else"
6: MOVI R2, 0            ; rama "else": resultado = 0
7: NOP                  ; fin
```

⚠️ **Caso límite a tener presente:** esta traducción usa `JN` (negativo estricto), así que cuando $a=b$ el resultado de `CMP` es 0 (ni negativo), y el programa cae en la rama **then**, ejecutando `resultado = a-b = 0`. Esto es *consistente* con la condición `a > b` original en C (si `a==b`, `a>b` es falso, y el resultado correcto según el pseudocódigo sería `0` por la rama `else`) — pero

como $a-b=0$ cuando $a=b$, la rama `then` da igualmente `resultado=0`. Es decir, en este caso particular ambas ramas coinciden numéricamente cuando $a=b$, así que la traducción es funcionalmente correcta a pesar de tomar la rama “equivocada” — vale la pena notarlo porque no siempre es tan afortunado: si las ramas hicieran cosas *distintas* además de asignar resultado, un `if` mal traducido con `JN` en lugar de `JN + JZ` combinados podría fallar en el caso de igualdad.

Traza de verificación — Caso 1: $a=15$, $b=9$ ($a>b$, debe tomar la rama *then*):

Paso	Instr.	Z	N	R0	R1	R2
1	MOVI R0,15	-	-	15	0	0
2	MOVI R1,9	-	-	15	9	0
3	CMP R0,R1 (15-9=6)	0	0	15	9	0
4	JN 6 ($N=0$, no salta)	0	0	15	9	0
5	SUB R2,R0,R1	0	0	15	9	6
6	JMP 7	0	0	15	9	6

Resultado: $R2=6=15-9$. Correcto.

Caso 2: $a=4$, $b=9$ ($a<b$, debe tomar la rama *else*): `CMP R0,R1` da $4-9=-5$, $N=1$, así que `JN 6` **sí** se toma, saltando directo a `MOVI R2,0`. Resultado: $R2=0$. Correcto.

✅ **Verificación:** se simularon ambos casos completos en la CPU de ISA-P. Caso 1 dio $R2=6$ (esperado $15-9=6$). Caso 2 dio $R2=0$ (esperado, rama *else*). También se probó el caso límite $a=b=7$: el flujo cae en la rama *then* (`JN` con $N=0$ no salta) y calcula $R2=7-7=0$, coincidiendo con lo discutido arriba.

Ejercicio 6 (Difícil) — Traducción de un `while` con recorrido de arreglo

Enunciado: traducir a ISA-P el siguiente fragmento, donde `arreglo` es un arreglo de 5 enteros ubicado en memoria a partir de la dirección `0x30`:

```

int suma = 0;
int i = 0;
while (i < 5) {
    suma = suma + arreglo[i];
    i = i + 1;
}

```

Estado inicial de memoria: `mem[0x30..0x34] = [3, 1, 4, 1, 5]`.

RESOLUCIÓN

Mapeo de variables: R0=`suma`, R1=`i`, R2=constante 5 (límite), R3=constante 0x30 (base del arreglo), R4=dirección calculada del elemento actual (`base+i`), R5=`arreglo[i]` leído, R6=constante auxiliar 1.

La condición `i < 5` se evalúa igual que en el Ejercicio 5: `CMP i,5` seguido de `JZ` para el caso de igualdad — pero acá se sale del loop cuando `i==5` (condición de salida, no de continuación), así que basta con `JZ` (Z=1 exactamente cuando `i==5`, que es el único valor de salida ya que `i` se incrementa de a 1 desde 0 — no hay riesgo de “saltarse” el 5 como en el problema de paridad visto en el Práctico 6, porque acá el paso es 1, no 2).

```

0: MOVI R0, 0      ; suma = 0
1: MOVI R1, 0      ; i = 0
2: MOVI R2, 5      ; límite = 5
3: MOVI R3, 0x30   ; base del arreglo
4: CMP  R1, R2     ; loop: ¿i == 5?
5: JZ   12         ; si i==5 -> fin
6: ADD  R4, R3, R1 ; R4 = base + i (dirección de arreglo[i])
7: LOAD R5, (R4)   ; R5 = mem[R4] = arreglo[i] (indirecto a registro)
8: ADD  R0, R0, R5 ; suma += arreglo[i]
9: MOVI R6, 1      ; R6 = 1 (constante; se recarga cada vuelta, sin costo funcional)
10: ADD R1, R1, R6 ; i += 1
11: JMP  4         ; volver a evaluar la condición
12: NOP           ; fin

```

Notar cómo el acceso `arreglo[i]` se traduce en **dos** pasos porque ISA-P no tiene un modo “base + índice” directo: primero se calcula la dirección con una suma explícita (paso 6, `ADD R4,R3,R1`), y recién después se usa el modo **indirecto a registro** (paso 7, `LOAD R5,(R4)`) para leer el contenido de esa dirección. Es exactamente la misma distinción dirección-vs-contenido remarcada en la guía temática y en el Práctico 6 (Ejercicio 3): R4 es una *dirección*, R5 es el *valor* almacenado ahí — nunca hay que confundirlos.

✅ **Verificación:** se codificaron las 13 instrucciones (direcciones 0-12) y se ejecutaron en la CPU de ISA-P con `mem[0x30..0x34] = [3,1,4,1,5]`. El programa se detuvo tras llegar a la instrucción NOP en la dirección 12, habiendo dado **5 vueltas completas** del loop ($i = 0,1,2,3,4$), y terminó con **R0 = 14** ($= 3+1+4+1+5$, verificado independientemente con `sum([3,1,4,1,5])` en Python) y **R1 = 5** (el valor de `i` que hizo cumplir la condición de salida, evaluada por sexta vez con `CMP R1,R2` dando $Z=1$ y disparando el `JZ 12`). El total de pasos de CPU ejecutados en la simulación (contando las 4 instrucciones de inicialización, las 5 vueltas completas del loop de 8 instrucciones cada una, la sexta evaluación de la condición que sale del loop, y la NOP final) fue 47, consistente con la traza.

Ejercicio 7 (Medio-Difícil) — Decodificación e interpretación de código máquina dado

Enunciado: el siguiente fragmento de 4 palabras de 16 bits fue extraído de la memoria de un programa en ISA-P, comenzando en la dirección 0. Decodificar cada palabra (opcode, campos, mnemónico con sus operandos) y explicar en una frase qué hace el fragmento como conjunto.

```
Dir 0: 0x1403
Dir 1: 0x1601
Dir 2: 0x3A98
Dir 3: 0xB000
```

RESOLUCIÓN

Palabra 0 — 0x1403 : binario `0001 010 00000011`. Opcode `0001` = MOVI. Campo Rd = `010` = R2. Campo Imm (9 bits) = `00000011` = 3. $\Rightarrow$ **MOVI R2, 3**.

Palabra 1 — 0x1601 : binario `0001 011 00000001`. Opcode `0001` = MOVI. Rd = `011` = R3. Imm = `00000001` = 1. $\Rightarrow$ **MOVI R3, 1**.

Palabra 2 — 0x3A98 : binario `0011 101 010 011 000`. Opcode `0011` = SUB. Formato de 3 registros: Rd = `101` = R5, Rs1 = `010` = R2, Rs2 = `011` = R3, últimos 3 bits sin usar = `000`. $\Rightarrow$ **SUB R5, R2, R3** (es decir, $R5 = R2 - R3$).

Palabra 3 — 0xB000 : binario `1011 0000 00000000`. Opcode `1011` = JN. Campo sin usar (4 bits) = `0000`. Campo Dir (8 bits) = `00000000` = 0. $\Rightarrow$ **JN 0**.

Interpretación conjunta: el fragmento carga R2=3 y R3=1, calcula $R5 = R2 - R3 = 3 - 1 = 2$ (positivo, así que N queda en 0), y termina con un salto condicional `JN 0` que **solo** se tomaría si esa resta hubiese dado negativo. Con estos valores concretos (3 y 1) el salto **no** se toma, y la ejecución sigue en la dirección 4. El patrón `SUB` seguido de `JN` a una dirección temprana del programa (aquí, 0) es característico de una **guarda de flujo controlada por signo**: por ejemplo, el inicio de un loop tipo “mientras R2 sea mayor o igual a R3, restar y volver a intentar” (un patrón de resta repetida / división por restas sucesivas), donde el salto hacia atrás se dispara justo cuando la resta se vuelve negativa (señal de que ya no se puede seguir restando).

✅ **Verificación:** las 4 palabras se generaron originalmente con el codificador de ISA-P a partir de `MOVI R2,3`, `MOVI R3,1`, `SUB R5,R2,R3`, `JN 0`, y luego se decodificaron de manera independiente con el decodificador (sin mirar el origen) — el resultado del decodificador coincidió exactamente con las cuatro instrucciones mencionadas arriba, confirmando que la lectura manual de campos (bit a bit) hecha en esta resolución es correcta.

Preguntas teóricas rápidas tipo parcial

Tapá la columna de respuesta y resolvé antes de mirar.

Pregunta	Respuesta
¿Por qué el banco de registros es más rápido que la memoria principal, si ambos almacenan bits de la misma forma lógica?	Por razones tecnológicas, no lógicas: (1) están físicamente pegados a la ALU, lo que reduce la distancia (y por lo tanto el tiempo) de la señal; (2) la memoria rápida es más cara de fabricar, así que se usa poca cantidad (los pocos registros) en el lugar donde más impacta el rendimiento.
¿Qué diferencia hay entre MAR y MBR/MDR, y en qué etapa del ciclo de instrucción se usa cada uno primero?	El MAR (Memory Address Register) contiene la <i>dirección</i> que se va a acceder; el MBR/MDR (Memory Buffer/Data Register) contiene el <i>dato</i> leído o a escribir en esa dirección. El primer uso de ambos ocurre en el Fetch: MAR se carga con el PC antes de leer memoria, y el dato leído (la instrucción) llega al MBR antes de copiarse al IR.
En una arquitectura load-store (RISC) puro, ¿por qué las instrucciones aritméticas nunca acceden directamente a memoria? ¿Qué ventaja da esto de cara al pipeline?	Porque el diseño obliga a traer los operandos a registros primero (con LOAD) y recién ahí operar; esto simplifica la Unidad de Control (menos combinaciones de “de dónde puede venir un operando”) y hace que cada etapa del ciclo de instrucción tienda a durar lo mismo entre instrucciones distintas — condición necesaria para un pipeline eficiente, ya que una etapa que a veces necesita un acceso a memoria extra (como en CISC) rompe la uniformidad de duración entre etapas.
En el modo indirecto a registro, si el registro base cambia entre dos ejecuciones sucesivas de la misma instrucción de LOAD, ¿qué pasa? ¿Y si el modo fuera directo a memoria?	En indirecto a registro, la instrucción (el código binario) no cambia, pero accede a una dirección distinta cada vez, porque la dirección efectiva depende del <i>contenido</i> del registro en ese momento — es justamente lo que permite recorrer un arreglo reutilizando la misma instrucción de LOAD dentro de un loop (Ejercicio 6). En directo a memoria, la dirección está fija dentro del código de la instrucción, así que siempre accede a la misma celda sin importar el estado de los registros — para acceder a otra dirección haría falta otra instrucción distinta (u otro programa).
¿Qué diferencia hay entre lógica cableada y lógica microprogramada respecto a qué hay que modificar para agregar una instrucción nueva al set?	En lógica cableada hay que rediseñar el circuito combinatorio/secuencial completo de la Unidad de Control (nuevas compuertas, nuevo diagrama de estados). En lógica microprogramada alcanza con escribir un nuevo microprograma (secuencia de microinstrucciones) en la ROM de Control Store, sin tocar el hardware de la UC — es la razón de ser de la microprogramación: flexibilidad de mantenimiento a costa de algo de velocidad.
Si una ISA tiene 12 instrucciones distintas y 8 registros, ¿cuántos bits	Opcode: $\lceil \log_2(12) \rceil = 4$ bits (verificación: $2^3=8 < 12$, no alcanza; $2^4=16 \geq 12$, sí alcanza, con 4 códigos de opcode sin usar). Registro: $\lceil \log_2(8) \rceil = 3$ bits (verificación: $2^3=8$, ex-

Pregunta	Respuesta
mínimos hacen falta para el campo de opcode y cuántos para cada campo de registro?	acto, sin desperdicio). Es exactamente el caso de ISA-P usado en todo este documento.

Resumen de técnicas usadas en este documento

Ejercicio	Técnica
1, 2	Traza completa del ciclo de instrucción con PC, IR, MAR, MBR y flags Z/N, distinguiendo qué instrucciones tocan memoria y cuáles no
2, 6	Contraste explícito entre direccionamiento directo a memoria (dirección fija en la instrucción) e indirecto a registro (dirección variable, necesaria para recorrer arreglos)
3, 4	Codificación campo por campo de un formato de instrucción con múltiples sub-formatos de ancho fijo (RISC), verificada por codificación+decodificación (round-trip)
5	Traducción de <code>if/else</code> usando <code>CMP</code> + salto condicional por signo (<code>JN</code>), con verificación explícita del caso límite de igualdad
6	Traducción de <code>while</code> con recorrido de arreglo: separación explícita entre <i>cálculo de dirección</i> (<code>ADD</code>) y <i>acceso al contenido</i> (<code>LOAD</code> indirecto), y condición de salida por igualdad exacta (<code>JZ</code>) cuando el paso del loop es 1
7	Decodificación inversa (bits → mnemónico) e interpretación del propósito del fragmento como patrón reconocible, no solo como secuencia de instrucciones aisladas

Para el examen

1. Frente a cualquier ISA nueva que te den en el parcial, lo primero es reconstruir su **tabla de formato** (qué bits son opcode, qué bits son cada operando) sumando anchos y comparando contra el ancho total declarado — exactamente como se hizo en la Sección 0 de este documento.
2. **Distinguí siempre modo de direccionamiento directo de indirecto a registro** por *de dónde sale la dirección* (fija en la instrucción vs. contenida en un registro que puede cambiar), no por la cantidad de accesos a memoria.
3. Al traducir un `if`, verificá el **caso límite de igualdad** entre operandos — es el error más común al elegir entre `JZ` / `JN` / combinaciones de ambos.

4. Al traducir un `while` con acceso a arreglo, **nunca combines el cálculo de la dirección y la lectura del valor en un solo paso mental** — sepáralos en dos instrucciones (una que calcula la dirección, otra que accede al contenido), tal como remarca también el Práctico 6.
5. Ante un ejercicio de decodificación, andá **campo por campo según el ancho declarado del formato** (nunca “a ojo”) y, si podés, verificá recodificando la instrucción que creés haber identificado — si no te da el mismo binario original, hay un error en la lectura de algún campo.